

Functional Block Identification in Circuit Design Recovery

Jacob Couch, Elizabeth Reilly, Morgan Schuyler, Bradley Barrett

Johns Hopkins University
Applied Physics Laboratory
Laurel, MD

Email: {jacob.couch, elizabeth.reilly, morgan.schuyler, bradley.barrett}@jhuapl.edu

Abstract—Design recovery is commonly conducted across many different platforms to gain knowledge about the underlying internals of a system. In this paper, a concept of segmentation and fuzzy matching is introduced to identify IP blocks within a design. Through this process, known IP blocks, especially in optimized ASIC and FPGA designs, can be identified within a netlist. Furthermore, these algorithms are computationally more efficient in comparison to the traditional subgraph isomorphism problem.

I. INTRODUCTION

Due to the increased use of both soft and hard Intellectual Property (IP) blocks within designs, the continued push to High Level Synthesis (HLS) development environments, and the verification of post-synthesis gate level netlists is not only uncommon, but challenging to conduct. Hardware designs that have been compiled to remove all annotations and hierarchy are rarely verified to ensure that the intended design is free from any non-intended logic, either malicious or benign.

This problem was investigated by DARPA through the TRUST project [1]. While there was excellent research conducted through this effort, it was primarily focused on detecting malicious modifications to a design. Although that is one use of design recovery techniques, the design recovery problem also routinely arises in compatibility testing, copyright investigations, and Trojan detection [2] [3] [4].

Many times in the design recovery process, custom Application Specific Integrated Circuits (ASICs) and Field Programmable Gate Arrays (FPGAs) are treated as black boxes in comparison to typical commodity hardware and software designs. This black box provides a gap in analyzing the entire system, though there are a few efforts that have focused on addressing this shortcoming. This work presents a technique to assist in this design recovery by identifying known blocks within a flattened and non-annotated FPGA or standard cell ASIC design. Once the blocks are identified and annotated, an engineer can work by hand through the remaining design to perform a complete design recovery. This research is not interested in steps that are required to get to this point. After the execution of this research, the design can then be further analyzed and simulated in other higher level tools such as simulators and emulators with stronger knowledge of the appropriate test vectors to exercise the portion of the design of interest.

Section II provides an overview of current design recovery techniques for flattened netlists. Section III outlines the algo-

rithms and techniques utilized in this research to advance the netlist recovery process. Section IV outlines the results from these algorithms. Section V provides an overview of future research and tools that can be enabled from this research. Throughout this paper, the authors seek to provide the foundation of segmentation and fuzzy matching that identifies blocks that may have been subject to non-deterministic optimizers, and to complete this analysis in a computationally reasonable time.

II. BACKGROUND OF HDL DESIGN RECOVERY

Substantial research has been conducted on restoring flattened netlists into a higher level analysis. One recent program in this field, the DARPA Integrity and Reliability of Integrated Circuits (IRIS), was tasked with determining the function of digital, analog and mixed signals integrated circuits [5].

The DARPA IRIS program expanded on the DARPA TRUST program to look into specific methods of design recovery within a netlist. Much of this work was on identifying exact replications of known circuits within the design through optimizations of the designs. Additional research in the field of HDL design recovery was focused on Finite State Machine (FSM) recovery and accelerated subgraph isomorphism solving, and behavioral pattern matching.

Shi et al. published an algorithm to identify FSMs from a flattened netlist [6]. This technique is based on identifying the cycles within the netlist which are associated with appropriate registers. These cycles are then evaluated to determine if they could be a FSM.

Li proposes a technique that involves matching sub-circuits against abstract library components and then performing data mining to aggregate these components into larger logic blocks [7]. This technique requires that the maximum clique be calculated for all comparisons to generate all signal correspondences.

Subramanyan, et. al., utilize the KCut algorithm for segmentation of the design graph, and then utilize aggregation, word propagation, module generation, and library matching to identify modules under the DARPA IRIS program [8].

In general, all of these mentioned techniques rely on utilizing subgraph isomorphism, and can become computationally unfeasible for modern designs.

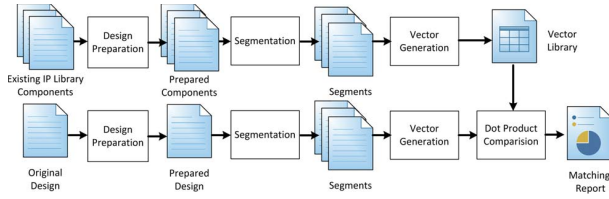


Fig. 1: Full transformation of a circuit design

III. APPROACH

This research looks primarily at the interconnection of various nodes within a circuit as the attributes. The goals of this research are to perform intelligent classification of regions of a netlist in order to reduce the size of a design that must be subject to blind design recovery efforts. This is in contrast to many existing approaches which rely on existing “golden designs” and/or exact subgraph matching. To accomplish this, two major algorithms are utilized to perform segmentation of the design followed by a fuzzy matching algorithm. The overview of the components in this approach is shown in Figure 1.

A. Design preparation

Before segmentation and matching can be conducted, the design must be prepared such that all logic and memory elements are broken into atomic units. These elements will become the nodes in the graph that represents the design. The nodes are also characterized into a limited character alphabet which contains nodes such as memory, logic, and external connections. Although this generalization is a lossy operation, it reduces the complexity of the problem for later analysis.

The design is also modified to identify all memory operations, and place an additional memory element after each memory element. This operation is conducted to ensure that for all segmentations there is a memory element on the boundary instead of leaving the indeterminism if the memory element should be segmented into cluster A or cluster B.

For all net interconnections within the design they are replaced with undirected edges in the graph due to the requirements of NCut. The choice of undirected edges is utilized as both the segmentation and matching algorithms selected expect undirected graphs. Furthermore, designs should be segmented if there are separate clock domains by the clock domain. Once this is completed, the remaining clock edges within the a single domain should be devalued.

B. Segmentation

Although there are no exact rules for hardware design, the continued presence of core based designs and HLS tools provide more and more designs that are based on clusters of highly interconnected logic and memory elements. These highly interconnected clusters are then sparsely connected either to a common bus or to other highly interconnected clusters [9]. This research is in contrast to most prior art that focuses on small cluster sizes. These highly interconnected larger cluster provide a more natural segmentation point that pairs well with the primitives provided by other IP functions.

Given a graph as described above, the goal is to partition the graph such that the clusters are relatively high density,

while the number of edges between clusters is sparse. The result is a number of smaller functional component designs, which may then be compared with other smaller known components. For graph partitioning, the authors employ the normalized cut (NCut) algorithm developed by Shi and Malik for image segmentation [10].

Assume that for nodes u, v in a given graph, the edge between them has a weight denoted by $w(u, v)$. Let A, B be a partition of the vertices of the graph. Then, the cut of A and B is

$$cut(A, B) = \sum_{u \in A, v \in B} w(u, v). \quad (1)$$

The cut between two sets measures the number of edges that need to be removed from the graph to completely disconnect A and B . Similarly, Shi and Malik define an association value for a set A with the entire vertex set, V , as the following:

$$assoc(A, V) = \sum_{u \in A, t \in V} w(u, t). \quad (2)$$

The association measures the total connection of set A within the given graph. Then, NCut is defined as

$$NCut(A, B) = \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(B, V)}. \quad (3)$$

The first term can be thought of as the proportion of set A 's connections in the graph that need to be removed in order to disconnect A from B . The second term is the same for set B . Minimizing this term results in a cut that considers both within cluster similarity and total dissimilarity of clusters in the graph. Though minimizing the Normalized Cut is NP-Complete, Shi and Malik provide an efficient algorithm to discover an approximate solution [10].

Because of the unknown optimal sizing of the clusters, multiple passes of the NCut algorithm are completed across the design. All the clusters that are derived from this algorithm are recorded and placed into a library for the matching algorithm. The library clusters are sourced from common IP sources. They were recompiled optimized with different parameters to introduce minor non-deterministic modifications to the test article. Clusters were also introduced into the library that were not part of the test design. This is done to simulate blind design recovery.

C. Matching

Once the design is segmented, the resulting subgraphs, also referred to as clusters, are compared to existing designs in a library in order to identify functional components within the larger original design. Figure 2 demonstrates this concept.

In its exact form, this is the Graph Isomorphism problem. In this paper, an approximate matching approach is taken by first embedding each graph into a low dimensional vector space (a lossy step) for comparison. This is desirable to allow for slight differences in designs due to compiler or programmer differences. Specifically, the authors use the concept of dot product representations for embedding. In 2010, Scheinerman and Tucker proposed this ‘vertices to vectors’ approach as a way of embedding the most relevant pieces of a graph’s connectivity into low dimensional vector space, allowing for

TABLE I: Segmentation Results

Partition	Cluster 1	Cluster 2	Cluster 3	Cluster 4	Cluster 5	Cluster 6	Cluster 7	Cluster 8	Cluster 9	Cluster 10	Cluster 11
Control Core A	0	72	0	0	0	0	0	0	9	0	0
Control Core B	22	47	0	52	0	0	0	0	2800	0	768
Memory A	0	0	0	0	0	0	0	0	0	0	0
FFT A	0	0	0	0	239	0	1	0	4	12	2420
Memory B	0	0	0	0	0	0	0	0	0	0	2
FFT B	0	0	486	0	0	172	1	0	2	1179	739
Memory C	0	0	0	0	0	0	1	0	0	0	1
AES A	0	0	0	7	0	0	1149	1	0	0	0
AES B	0	0	0	927	0	0	24	10	0	0	2
AES C	3	0	0	1	0	0	0	1255	0	0	1
AES D	915	0	0	3	0	0	0	22	17	0	32

The large test design has 11 subdesigns totaling 14972 vertices. Dark blocks indicate segments where the majority of the original subdesign landed within the segmentation. Lighter blocks indicate partial matches.

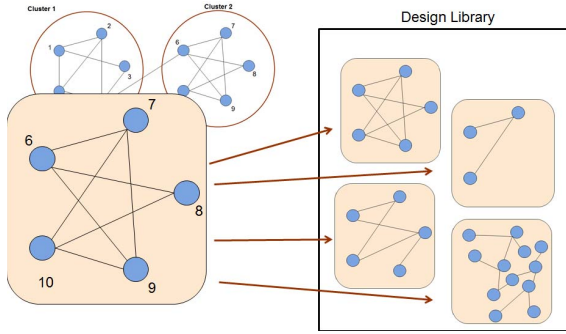


Fig. 2: After segmentation, each cluster is compared with every design in a design library to identify known functional components.

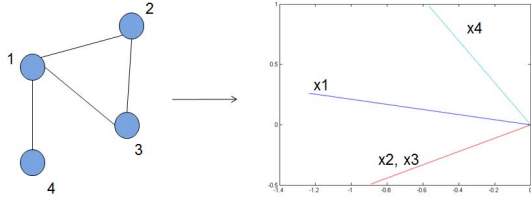


Fig. 3: In this dot product representation example, each vertex in the graph is assigned a vector in $\mathbf{R}^2$. Notice that vertices 2 and 3 are assigned the same vector since they play the same role in the graph. The vector for vertex 4 is almost orthogonal to x_2 and x_3 since the only thing 2,3,4 have in common is vertex 1, whose vector is between the other vectors (and largest).

geometric analysis methods [11]. Given a graph, each vertex i is assigned a vector x_i in $\mathbf{R}^d$ for a user selected dimension d such that, for distinct vertices i and j , the dot product $x_i \cdot x_j$ is approximately equal to the weight of the edge between i and j . For a vertex i , the magnitude of x_i reflects how connected i is in the graph. The angle between two vectors corresponds to the connectivity of the vertices. For example, two vertices with little in common will be assigned vectors that are roughly orthogonal. For a simple example, see Figure 3. Scheinerman and Tucker propose an iterative algorithm to calculate the dot product representation of a graph in [11]. This algorithm was implemented in MATLAB with dimension equal to 50.

After embedding into $\mathbf{R}^d$, two sets of vectors are compared. In this paper, the Procrustean transformation is used to align the sets of vectors and find the norm of the difference

between the two sets [12].

By embedding each sub-circuit into a low dimensional space and using geometric analysis methods, the authors were able to analyze a total circuit of over 10000 nodes with sub-circuits of over 1000 nodes. Not only is this an improvement over existing techniques, but it also allows for fuzzy design matching to account for compiler and minor design differences.

IV. RESULTS

For the purpose of these results, a test design was constructed that contained two different AES cores, two different FFT cores, and two small microprocessor control blocks. These cores were pulled from publicly available IP libraries. This combined design resulted in over 14000 nodes in the design. Additional cores were added to the design library including other types of FFT and AES cores, along with other microprocessors to provide additional comparison vectors for the matching algorithm.

A. Segmentation

The NCut algorithm was applied to the overall design in order to identify relevant smaller functional elements in the design. The largest design considered has 14972 vertices and 62527 edges. A mid-2015 Macbook Pro computer with a 2.8 GHz core i7 processor with 16 GBs of RAM was used for all experiments. This design ran in 5.8 seconds.

Table I shows results of running the NCut algorithm on the large design for 11 clusters. Notice that the AES blocks mostly segment into their own individual clusters. Stray vertices are not of concern as the matching step is fuzzy. The other large pieces are two control cores and two FFTs. Each control core mostly has its own segment (clusters 2 and 9), though core B loses a significant 768 nodes to cluster 11, as well. The largest segments of the FFTs correspond to clusters 10 and 11. The FFT subdesigns breaks into multiple segments. Each one has ‘rogue segments’ in clusters 3, 5, or 6 indicating that FFTs may inherently be composed of smaller functional units. In this case, the smaller functional FFT units should be included in the matching design library instead of the FFT itself. Nonetheless, this segmentation is sufficient for the matching algorithm to extract the main pieces of the overall design.

B. Matching

The output of the previous step is several smaller clusters, which are then compared to designs in a library. Embedding

TABLE II: Co-similarity of designs

ID	True	Calculated	Match
1	AES D	AES	0.11994
2	Control Core A	Memory	0.34893
3	FFT B	AES	0.44194
4	AES B	AES	0.11806
5	FFT A	Memory	0.57913
6	FFT B	Memory	0.33189
7	AES A	AES	0.065193
8	AES C	AES	0.066218
9	Control Core B	Control Core	0.083697
10	FFT B	AES	0.080711
11	FFT A	FFT	0.057055

Matching against the library of known blocks. Here the recovered sub-circuits are compared against a slightly reduced library of possible sub-circuit designs.

each cluster into $\mathbf{R}^d$ is the most time consuming step of the process. In the specific matching described in this paper, the dimension $d = 50$ was utilized to provide sufficient information about each partition, while reducing the computational complexity. Calculating the dot product representation of a cluster with 1423 vertices and 3975 edges takes roughly 5.5 seconds using MATLAB.

The test library contains 16 designs containing all the known functional elements in the overall design, and designs similar to the known designs to test the robustness of the algorithm. The designs in the library have between 117 and 4608 vertices and between 371 and 10786 edges after the initial design preparations.

An initial test of the matching algorithm showed that each subdesign matches with itself in the design library. This was a necessary step since the dot product representation algorithm is lossy and iterative. Thus, results may vary slightly between runs of the same design. Through multiple iterations, it was discovered that the difference across runs is negligible and using dimension 50 still allows for ample differentiation between designs.

Next, the authors turned toward the proposed end to end segmentation and matching algorithm. Each cluster in Table I was run through the proposed matching algorithm. The matching algorithm proved to be robust to incorrect vertex groupings. For example, cluster 11 consists primarily of FFT A with significant contributions from Control Core B and FFT B. Nonetheless, the matching algorithm correctly identifies cluster 11 as being most similar to the FFT A subdesign. Similarly, cluster 9 was identified as Control Core B, and clusters 1, 4, 7, 8 were all identified as AES blocks. Thus, all the major elements of the design were identified, except for FFT B. Clusters 2, 3, 5, and 6 are all relatively small clusters and did not match well with anything. In fact, this is what would be expected to happen since those clusters are small pieces broken off of the FFT blocks. This is where a reverse engineer would analyze the overall design to realize that is where the small unmatched blocks are coming from. The results of the matching are shown in Table II. Although there are mismatches in the final table, the majority are a function of the small sizes of the memory blocks that are subsets of the control cores and the FFT. Overall, these results are promising and demonstrate how robust and effective the proposed matching algorithm can be.

V. FUTURE WORK

Although this work is still in its infancy, it shows that both segmentation and fuzzy matching can be utilized to assist in the design recovery process. By capturing common components from various IP libraries, a reasonably sized library can be constructed to assist in the initial triage of future unknown designs. The machine time that is required to generate the library, and to do the corresponding matches is reasonable for most moderate size FPGA and ASIC designs of hundreds of thousands of nodes to millions of nodes. With the continued use of precompiled IP blocks, it is reasonable to expect the library to contain thousands to tens of thousands of vectors to match against while still having acceptable execution time. Furthermore, these vectors can be further evaluated and matched utilizing many methods that are common to software design recovery processes without revisiting the graph sub-isomorphism problem.

VI. CONCLUSION

Although substantial research exists on techniques to identify known or unknown sub-circuits within a design, they are generally focused on exact sub-circuit matches. These existing techniques are typically utilized for test vector generation and golden design verification. This research shows the viability of utilizing fuzzy matching by reducing the alphabet of the nodes, providing a reasonable segmentation method, and vectorizing the elements to provide a linear matching algorithm. The techniques shown above are aimed at assisting blind design recovery efforts by reducing the human time that is required to gain an understanding of the design while also providing reasonable computation time for the analysis.

REFERENCES

- [1] DARPA, "Trusted Integrated Circuits (TRUST)."
- [2] G. R. Ignatin, "Let the Hackers Hack: Allowing the Reverse Engineering of Copyrighted Computer Programs to Achieve Compatibility," *University of Pennsylvania Law Review*, no. 5, pp. 1999–2050.
- [3] J. Kumagai, "Chip detectives [reverse engineering]," *IEEE Spectrum*, no. 11, pp. 43–48.
- [4] M. Tehranipoor and F. Koushanfar, "A survey of hardware Trojan taxonomy and detection."
- [5] K. Bernstein, "Integrity and Reliability of Integrated Circuits (IRIS)."
- [6] Y. Shi, C. W. Ting, B. H. Gwee, and Y. Ren, "A highly efficient method for extracting FSMs from flattened gate-level netlist," in *ISCAS 2010 - 2010 IEEE International Symposium on Circuits and Systems: Nano-Bio Circuit Fabrics and Systems*, 2010, pp. 2610–2613.
- [7] W. Li, Z. Wasson, and S. A. Seshia, "Reverse engineering circuits using behavioral pattern mining," in *2012 IEEE International Symposium on Hardware-Oriented Security and Trust*. IEEE, Jun., pp. 83–88.
- [8] P. Subramanyan, N. Tsiskaridze, W. Li, A. Gascon, W. Y. Tan, A. Tiwari, N. Shankar, S. A. Seshia, and S. Malik, "Reverse Engineering Digital Circuits Using Structural and Functional Analyses," *IEEE Transactions on Emerging Topics in Computing*, no. 1, pp. 63–80, Mar.
- [9] T. Frangieh and P. Athanas, "A design assembly framework for FPGA back-end acceleration," in *2012 International Conference on Reconfigurable Computing and FPGAs*. IEEE, Dec., pp. 1–6.
- [10] J. Shi and J. Malik, "Normalized cuts and image segmentation," *Pattern Analysis and Machine Intelligence, IEEE*
- [11] E. Scheinerman and K. Tucker, "Modeling graphs using dot product representations," *Comput Stat*, vol. 25, 2010. [Online]. Available: <http://link.springer.com/article/10.1007/s00180-009-0158-8#page-1>
- [12] R. Horn and C. Johnson, *Matrix Analysis*. Cambridge University Press, 1985.